Principles of Programming Languages, 2024.09.03

Important notes
- Total available time: 2h (*multichance* students do not need to solve Exercise 3).
- You may use any written material you need, and write in English or in Italian.
- You cannot use electronic devices during the exam: every phone must be turned off and kept on your table.
- You cannot use library functions not covered in class in your code.

Exercise 1, Scheme (11 pts)

Consider the *delay/force* construct used to implement call-by-need. Define an extension of the construct by adding the concept of a **master promise**: other promises could be created and linked to a master promise.
When the master promise is **evaluated**, all its linked promises are **evaluated** too.
In particular, we need the following constructs:

1. *(delay-master <expression>)* to create a master promise, and
2. *(linked-delay <master> <expression>)* to create a linked promise.

Here's the implementation of the *delay/force* construct seen in class:

```
(struct promise
  (proc    ; thunk or value
   value?) ; already evaluated?
  #:mutable)

(define-syntax delay
  (syntax-rules ()
    ((_ (expr ...))
     (promise (lambda ()
                (expr ...))
              #f))))
```

```
(define (force prom)
 (cond
  ; is it already a value?
  ((not (promise? prom)) prom)
  ; is it an evaluated promise?
  ((promise-value? prom) (promise-proc prom))
  (else
    (set-promise-proc! prom
      ((promise-proc prom)))
    (set-promise-value?! prom #t)
    (promise-proc prom))))
```

Exercise 2, Haskell (11 pts)

Consider the following data structure, implementing a "double content" list:

```
data Llist b a = Nod a b (Llist b a) | Nul deriving (Show, Eq)
```

1) Make *Llist* an instance of Functor and Foldable.

2) We cannot make it an instance of Applicative, as it is: why? Make *Llist Bool* an instance of Applicative.

Exercise 3, Erlang (11 pts)

Consider the delay-force construct presented in class to implement call-by-need in Scheme.
We want to implement an Erlang version, where promises are Erlang processes.
The interface of the construct is the following:

- *delay(<function>)*, where <function> is a thunk -- we cannot pass arbitrary code as in Scheme, so we need a thunk: why?
- *force(<Pid>)*, where <Pid> is the process id of the promise.

Solutions

Ex 1

```
(struct promise-master promise
        ((link #:mutable)))

(define-syntax delay-master
  (syntax-rules ()
    ((_ (expr ...)) (promise-master (lambda ()
                                      (expr ...))
                                    #f '()))))

(define-syntax linked-delay
  (syntax-rules ()
    ((_ master (expr ...))
     (let ((p (promise (lambda ()
                         (expr ...))
                       #f)))
       (when (promise-master? master)
           (set-promise-master-link! master (cons p (promise-master-link master))))
       p))))

(define (force prom)
  (cond
    ((not (promise? prom)) prom)
    ((promise-value? prom) (promise-proc prom))
    (else
      (set-promise-proc! prom
                         ((promise-proc prom)))
      (set-promise-value?! prom #t)
      (when (promise-master? prom)
        (for-each (lambda (x)
                    (force x))
                  (promise-master-link prom)))
      (promise-proc prom))))
```

Ex 2

```
instance Functor (Llist b) where
    fmap f Nul = Nul
    fmap f (Nod x y s) = Nod (f x) y (fmap f s)

instance Foldable (Llist b) where
    foldr f z Nul = z
    foldr f z (Nod x y s) = f x (foldr f z s)

Nul +++ y = y
y +++ Nul = y
(Nod x y s) +++ s' = Nod x y (s +++ s')

tcconcat :: Foldable t => t (Llist b a) -> Llist b a
tcconcat = foldr (+++) Nul

-- We cannot implement pure, because we need a value of type b for it
instance Applicative (Llist Bool) where
    pure x = Nod x True Nul   -- in this case we know that b = Bool, and we chose True as the default value
    fs <*> xs = tcconcat $ fmap (\f -> fmap f xs) fs
```

Ex 3

```
% this takes a thunk, because Erlang's macro system is too weak
delay(F) ->
    spawn(?MODULE, promise, [F, false]).

promise(F, B) ->
    receive
        {value, Pid} ->
            if
                B -> Pid ! {self(), F}, promise(F, B);
                true ->
                        V = F(),
                        Pid ! {self(), V},
                        promise(V, true)
            end
    end.

force(Pid) ->
    Pid ! {value, self()},
    receive
        {Pid, V} -> V
    end.
```